Plan Maestro Unificado y Arquitectura Integral: InvestAI

A continuación se presenta la consolidación exhaustiva de la planificación de InvestAI. Este documento integra sinérgicamente los objetivos originales relevados en el trabajo de campo y la planificación estratégica de la Etapa 1 con la arquitectura de grado institucional, la remediación de auditoría y los módulos de computación cuantitativa en vivo implementados recientemente.

El plan está organizado bajo una estructura jerárquica estricta de Capítulos (hitos evolutivos del sistema), Módulos (unidades de dominio, infraestructura, transporte y datos) y Fases de Implementación, detallando su estado actual (IMPLEMENTADO o PENDIENTE), bases matemáticas, modelos de datos, estándares de seguridad y suites de prueba asociadas.
MAPA GLOBAL DE EVOLUCIÓN

┌──┐

│ [IMPLEMENTADO] CAPÍTULO 1: Fundaciones del Dominio Cuantitativo y Arquitectura Base │

│ ├─ Módulo 1.1: Álgebra Lineal Cuantitativa y Regularización Espectral │

│ ├─ Módulo 1.2: Motores de Optimización de Carteras (SLSQP y Risk Parity) │

│ ├─ Módulo 1.3: Máquina de Estados Finitos (FSM) y Microestructura de Fricciones (BYMA) │

│ ├─ Módulo 1.4: Motor de Backtesting y Simulación Estocástica │

│ └─ Módulo 1.5: Persistencia Relacional y Capa de Exposición Inicial │

├──┤

│ [IMPLEMENTADO] CAPÍTULO 2: Auditoría Integral y Diagnóstico Técnico (AUDIT_REPORT.md) │

│ ├─ Módulo 2.1: Diagnóstico de Arquitectura, Contratos y Seguridad en Transporte │

│ └─ Módulo 2.2: Diagnóstico Numérico Cuantitativo y Evaluación de Pruebas │

├──┤

| [IMPLEMENTADO] CAPÍTULO 3: Remediación Técnica y Hardening Post-Auditoría
|

| ├─ Módulo 3.1: Seguridad Defensiva y Sanitización Inmediata (SEC-01, SEC-02, ARQ-01)
|

| ├─ Módulo 3.2: Estabilización Numérica y Mitigación Adversarial (NUM-01 a NUM-04)
|

| ├─ Módulo 3.3: Modernización de la Suite de Pruebas (TST-01) |

| ├─ Módulo 3.4: Desacoplamiento Arquitectónico y Universal Envelope (ARQ-02, ARQ-03)
|

| └─ Módulo 3.5: Observabilidad y Telemetría de Producción (OBS-01, OBS-02) |

├──┤

| [IMPLEMENTADO] CAPÍTULO 4: Conectividad Externa, Ingesta en Vivo y Datos de Mercado
|

| ├─ Módulo 4.1: Contratos Canónicos y Calculadora FX MEP/CCL con Microestructura Bid-Ask |

| ├─ Módulo 4.2: Conectores Asíncronos, Resiliencia y Blindaje SSRF Perimetral |

| ├─ Módulo 4.3: Persistencia Relacional, Deduplicación Delta y Caché Volátil Monótono
|

| └─ Módulo 4.4: Controladores REST (Universal Envelope) e Integración con Backtesting
|

├──┤

| [PENDIENTE] CAPÍTULO 5: Seguridad Criptográfica, Autenticación y Control de Acceso (SEC-03) |

| ├─ Módulo 5.1: Modelo de Identidad, Hashing Criptográfico (Argon2id/Bcrypt) y Secretos
|

| ├─ Módulo 5.2: Motor de Tokens Criptográficos (JWT / OAuth2 Bearer) |

| ├─ Módulo 5.3: Dependencias de Seguridad, RBAC y Aislamiento Anti-IDOR/BOLA
|

│ └─ Módulo 5.4: Endpoints REST de Autenticación y Suite de Pruebas Herméticas
│

├──┤

│ [PENDIENTE] CAPÍTULO 6: Ingesta de Reportes Locales y Pipeline de Calidad de Datos (Campo) │

│ ├─ Módulo 6.1: Parser Desacoplado de Extractos y Reportes de ALyCs (CSV / Excel)
│

│ ├─ Módulo 6.2: Pipeline de Validación, Normalización y Tratamiento Contable (EPS <= 0)
│

│ └─ Módulo 6.3: Integración Transaccional y Conciliación Automática de Tenencias
│

├──┤

│ [PENDIENTE] CAPÍTULO 7: Motores Cuantitativos Avanzados, Señales y Valuación Mark-to-Market │

│ ├─ Módulo 7.1: Indicadores Técnicos y Generador de Señales Cuantitativas │

│ ├─ Módulo 7.2: Valuación Patrimonial Dinámica y PnL Multimoneda (TWR / MWR)
│

│ └─ Módulo 7.3: Rebalanceo por Bandas de Tolerancia Dinámicas (Tolerance-Band Rebalancing) │

├──┤

│ [PENDIENTE] CAPÍTULO 8: Ciclo de Vida de Base de Datos y Automatización en Background │

│ ├─ Módulo 8.1: Versionado y Migraciones Estructuradas con Alembic │

│ ├─ Módulo 8.2: Schedulers de Ingesta Bursátil Programada y Bloqueo Concurrente
│

│ └─ Módulo 8.3: Monitoreo Institucional y Métricas RED (Prometheus / Grafana) │

├──┤

│ [PENDIENTE] CAPÍTULO 9: Frontend Dashboard Reactivo (React + Vite + TypeScript)

        - Archivo: `backend/app/domain/services/numerical_stability.py`.
        - Algoritmo safe_cholesky:
            - Simetrización explícita: Sigma_sym = 0.5 * (Sigma + Sigma^T).
            - Adición iterativa de jitter diagonal: Sigma_reg = Sigma_sym + lambda * I, lambda en [10^-6, 10^-5, 10^-4, 10^-3, 10^-2]
            - Fallback de proyección de Higham: si el jitter diagonal no logra la descomposición L L^T, se calcula la descomposición espectral V Lambda V^T truncando autovalores negativos (Lambda_ii^+ = max(Lambda_ii, 10^-8)), reconstruyendo una matriz estrictamente definida positiva.
        - Funciones complementarias: evaluación de radio espectral, número de condición mediante SVD (matrix_condition_number) y cálculo de matriz semidefinida positiva más cercana.

[IMPLEMENTADO] Módulo 1.2: Motores de Optimización de Carteras (SLSQP y Risk Parity)

- **Objetivo:** Resolver problemas de asignación de capital deterministas bajo restricciones convexas de no-negatividad y presupuesto total.
- **Fases de Implementación:**
    - **Fase 1 — Portafolio de Varianza Mínima Global (Global Minimum Variance - GMV):**
        - Archivo: `backend/app/domain/services/portfolio_optimizer.py`.
        - Formulación de optimización cuadrática: min_w (1/2) w^T Sigma w s.a. sum_{i=1}^N w_i = 1, 0 <= w_i <= 1 para todo i
        - Resolución numérica mediante scipy.optimize.minimize(method="SLSQP") con punto inicial equiponderado w_0 = [1/N, ..., 1/N]^T.
    - **Fase 2 — Paridad de Riesgo / Equal Risk Contribution (ERC):**
        - Archivo: `backend/app/domain/services/portfolio_optimizer.py`.
        - Formulación convexa logarítmica de Spinu: min_y (1/2) y^T Sigma y - sum_{i=1}^N ln(y_i) s.a. y_i > 0
        - Gradiente analítico suministrado: grad f(y) = Sigma y - 1/y
        - Resolución mediante scipy.optimize.minimize(method="L-BFGS-B"). Normalización final canónica: w_i = y_i / sum_j y_j.

[IMPLEMENTADO] Módulo 1.3: Máquina de Estados Finitos (FSM) y Microestructura de Fricciones

- **Objetivo:** Modelar transaccionalmente el ciclo de vida de órdenes bursátiles y computar con exactitud aranceles, comisiones e impuestos del mercado argentino.
- **Fases de Implementación:**
    - **Fase 1 — FSM Transaccional de Órdenes:**
        - Archivo: `backend/app/domain/models/order.py`.
        - Estados: CREATED, VALIDATED, SUBMITTED, FILLED, REJECTED, CANCELLED.
        - Matriz determinista de transiciones:
            - CREATED → VALIDATED o REJECTED.
            - VALIDATED → SUBMITTED o CANCELLED.
            - SUBMITTED → FILLED o REJECTED.
        - Invariante de estados terminales: bloqueo absoluto de mutación tras alcanzar FILLED, REJECTED o CANCELLED.
    - **Fase 2 — Motor de Fricciones y Liquidación Local (BYMA / ALyC):**
        - Archivo: `backend/app/domain/services/market_frictions.py`.
        - Modelado detallado:
            - Comisión de ALyC porcentual sobre monto nominal, sujeta a comisión mínima fija (min_fee).
            - Impuesto al Valor Agregado (IVA): 21.00% liquidado estrictamente sobre la comisión cobrada por el intermediario.
            - Derechos de mercado de BYMA: tasa de negociación y liquidación bursátil.
        - Diferenciación de flujos de caja: incremento del costo de adquisición para compras (BUY) y deducción sobre el ingreso neto a liquidar para ventas (SELL).

[IMPLEMENTADO] Módulo 1.4: Motor de Backtesting y Simulación Estocástica

- **Objetivo:** Simular la evolución temporal de carteras bajo rebalanceos periódicos y computar métricas paramétricas y no paramétricas de riesgo y retorno.
- **Fases de Implementación:**
    - **Fase 1 — Simulador de Portafolios Paso a Paso:**
        - Archivo: `backend/app/domain/services/backtest_engine.py`.
        - Recorrido secuencial a través de series cronológicas de precios de cierre.
        - Rebalanceo periódico (diario, semanal, mensual o por bandas de tolerancia).
        - Cálculo integral de métricas: Retorno Total Acumulado, CAGR, Volatilidad Anualizada, Ratios de Sharpe y Sortino, Máximo Drawdown

(MDD), Value-at-Risk histórico (VaR 95%, VaR 99%) y Conditional Value-at-Risk (CVaR).
    - **Fase 2 — Generador Estocástico Browniano Geométrico (GBM):**
        - Generación sintética multivariada acoplada originalmente al endpoint: S_i(t + Delta t) = S_i(t) exp((mu_i - 0.5 * sigma_i^2) Delta t + sigma_i sqrt(Delta t) Z_i)
        - Correlación cruzada entre activos inyectada mediante descomposición de Cholesky de matrices de correlación empíricas.

[IMPLEMENTADO] Módulo 1.5: Persistencia Relacional y Capa de Exposición Inicial

- **Objetivo:** Almacenamiento inicial de entidades en base de datos relacional y exposición de endpoints básicos en FastAPI.
- **Fases de Implementación:**
    - **Fase 1 — Modelado Relacional (SQLAlchemy):**
        - Modelos: PortfolioModel, HoldingModel, OrderModel.
        - Configuración asíncrona con AsyncSession y motor aiosqlite.
    - **Fase 2 — Endpoints HTTP Base y Suite de Testing Original (137 Pruebas):**
        - Routers en `backend/app/api/v1/endpoints/`: portfolio.py, optimization.py, backtest.py.
        - Suite construida sobre unittest.TestCase y unittest.IsolatedAsyncioTestCase dentro de `backend/tests/unit` y `backend/tests/integration`.
        - Verificación: 137 tests ejecutados al 100% en verde.

CAPÍTULO 2: Auditoría Integral y Diagnóstico Técnico

Estado: Completado. Auditoría adversarial exhaustiva sobre la totalidad del código y emisión formal del documento docs/AUDIT_REPORT.md.

[IMPLEMENTADO] Módulo 2.1: Diagnóstico de Arquitectura, Contratos y Seguridad en Transporte

- **Objetivo:** Identificar acoplamientos indeseados, vulnerabilidades perimetrales y debilidades de tipado en la capa de transporte.
- **Fases de Implementación:**
    - **Fase 1 — Identificación de Fugas de Dominio (Transport Leaks):**
        - Detección de lógica algorítmica pesada (_generate_gbm_market_prices) dentro del controlador HTTP backtest.py.

        - Detección de generación de retornos sintéticos (_generate_synthetic_returns, _prepare_quantitative_inputs) dentro de optimization.py.
        - Bypass de modelos de dominio mapeando modelos ORM directamente a DTOs en portfolio_service.py.
    - **Fase 2 — Evaluación de Contratos de Entrada y Transporte:**
        - Ausencia del patrón Universal Envelope (APIResponse[T] / APIErrorResponse).
        - Omisión de extra="forbid" en modelos Pydantic v2, exponiendo al sistema a ataques de inyección de parámetros (mass assignment).
        - Entidades mutables sin frozen=True en modelos críticos de portafolios y órdenes.
    - **Fase 3 — Detección de Brechas Perimetrales (SEC-01, SEC-02, SEC-03):**
        - SEC-01: Configuración permisiva de CORS con comodín allow_origins=["*"] junto con allow_credentials=True.
        - SEC-02: Fuga de trazas de excepción interna (str(exc)) en respuestas HTTP 500 y exposición de cadenas de conexión a base de datos en GET /.
        - SEC-03: Autenticación simulada en deps.py basada en la lectura no validada del encabezado arbitrario X-User-Id (vulnerabilidad a IDOR/BOLA).

[IMPLEMENTADO] Módulo 2.2: Diagnóstico Numérico Cuantitativo y Evaluación de Pruebas

- **Objetivo:** Catalogar singularidades matemáticas, fallos de convergencia algorítmica y obsolescencia en la infraestructura de testing.
- **Fases de Implementación:**
    - **Fase 1 — Diagnóstico de Singularidades Matemáticas (NUM-01, NUM-02):**
        - NUM-01: Divergencia del gradiente analítico grad f(y) = Sigma y - 1/y en Spinu ERC ante activos sin volatilidad (sigma^2 <= 10^-12), induciendo valores -infinito que invalidan el solver.
        - NUM-02: Falta de validación de finitud (NaN/Inf) en vectores de solución devueltos por SLSQP y L-BFGS-B ante fallos silenciosos de convergencia.
    - **Fase 2 — Diagnóstico de Fricciones y Manejo de Series (NUM-03, NUM-04):**
        - NUM-03: Intersección estricta de fechas (set.intersection) en el motor de backtesting, contrayendo la ventana temporal al activo más joven y degradando la optimización a equiponderación.
        - NUM-04: Fricciones en ventas residuales con comisiones mínimas superiores al monto nominal provocando saldos netos negativos que drenan la caja.
    - **Fase 3 — Diagnóstico de Infraestructura de Pruebas y Observabilidad (TST-01, OBS-01, OBS-02):**

        - TST-01: Dependencia exclusiva de unittest, ausencia de conftest.py, advertencias de recursos por conexiones SQLite abiertas (ResourceWarning) y uso de constantes deprecadas de Starlette.
        - OBS-01 y OBS-02: Ausencia de correlación distribuida por request_id, falta de métricas de latencia de backend y endpoints de salud estáticos sin comprobación real de la base de datos.

CAPÍTULO 3: Remediación Técnica y Hardening Post-Auditoría

Estado: Completado. Ejecución sistemática de las 5 fases de remediación planificadas en la auditoría, elevando la suite de 137 a 159 tests al 100% en verde.

[IMPLEMENTADO] Módulo 3.1: Seguridad Defensiva y Sanitización Inmediata (SEC-01, SEC-02, ARQ-01)

- **Objetivo:** Erradicar vulnerabilidades de transporte perimetral y blindar la validación de entrada de todos los esquemas Pydantic v2.
- **Fases de Implementación:**
    - **Fase 1 — Endurecimiento Perimetral y Sanitización de Respuestas (`backend/app/main.py`):**
        - Eliminación del comodín "*" en CORS; fijación estricta de orígenes permitidos (http://localhost:5173, http://127.0.0.1:5173, etc.).
        - Sanitización del manejador global HTTP 500: registro de excepciones en logs y respuesta pública opaca ("An internal server error occurred.").
        - Sanitización de GET /: remoción de cadenas de conexión y exposición de metadatos seguros de la aplicación.
    - **Fase 2 — Esquemas Hardened con extra="forbid" (`backend/app/schemas/`):**
        - Inclusión de ConfigDict(extra="forbid", str_strip_whitespace=True) en esquemas de portafolios, optimización, backtesting, rebalanceo e ingesta.
    - **Fase 3 — Verificación y Testing (140 Tests en Verde):**
        - Pruebas de rechazo de campos extra en esquemas DTO y validación de respuestas HTTP 422 ante payloads con campos espurios. Total: 140 tests en verde.

[IMPLEMENTADO] Módulo 3.2: Estabilización Numérica y Mitigación Adversarial (NUM-01 a NUM-04)

- **Objetivo:** Blindar el optimizador de carteras ante singularidades numéricas y proteger la integridad de las series temporales y de la caja en el simulador.
- **Fases de Implementación:**

- **Fase 1 — Aislamiento de Activos Planos en Spinu ERC (NUM-01):**
  - Archivo: `backend/app/domain/services/portfolio_optimizer.py`.
  - Aislamiento de activos con varianza nula (sigma^2 <= 10^-12). Resolución de la optimización sobre el subespacio invertible de activos volátiles asignando peso cero a los activos planos. Activación de equiponderación si todos los activos carecen de volatilidad.
- **Fase 2 — Validación de No-Finitud y Fallbacks Seguros en Solvers (NUM-02):**
  - Validación rigurosa de convergencia y finitud (np.all(np.isfinite(x))). Activación determinista de fallbacks seguros: equiponderación (1/N) para SLSQP y ponderación por inversa de volatilidad para L-BFGS-B ante fallos numéricos.
- **Fase 3 — Unión Temporal con Imputación Segura (NUM-03):**
  - Archivo: `backend/app/domain/services/backtest_engine.py`.
  - Sustitución de intersección de fechas por unión temporal completa con imputación de retorno cero previa al listado de activos jóvenes, preservando la ventana de optimización.
- **Fase 4 — Piso No Negativo en Liquidación de Ventas Residuales (NUM-04):**
  - Archivo: `backend/app/domain/services/market_frictions.py` y `backtest_engine.py`.
  - Acotamiento inferior del ingreso neto en órdenes de venta: net_total = max(0.0, effective_amount - fricciones) evitando que comisiones mínimas generen flujos negativos de caja.
- **Fase 5 — Verificación y Testing (146 Tests en Verde):**
  - Incorporación de pruebas numéricas adversariales. Total: 146 tests pasando en verde.

[IMPLEMENTADO] Módulo 3.3: Modernización de la Suite de Pruebas (TST-01)

- **Objetivo:** Migrar la ejecución a pytest-asyncio, implementar un arnés global de fixtures y eliminar fugas de descriptores en SQLite.
- **Fases de Implementación:**
  - **Fase 1 — Parametrización Formal de Pytest (`backend/pytest.ini`):**
    - Configuración de asyncio_mode = auto, testpaths = tests, registro de marcadores unit e integration, y supresión de advertencias de Starlette.
  - **Fase 2 — Arnés Global de Fixtures (`backend/tests/conftest.py`):**
    - Fixtures test_engine (alcance sesión con StaticPool y disposición controlada), db_session (alcance función con rollback automático) y async_client (httpx.AsyncClient sobre ASGITransport).
  - **Fase 3 — Saneamiento de Fugas y Deprecaciones:**
    - Migración a status.HTTP_422_UNPROCESSABLE_CONTENT (RFC 9110) y cierre explícito de conexiones en asyncTearDown, erradicando el warning ResourceWarning: unclosed database.

[IMPLEMENTADO] Módulo 3.4: Desacoplamiento Arquitectónico y Universal Envelope (ARQ-02, ARQ-03)

- **Objetivo:** Desacoplar algoritmos estocásticos de la capa HTTP y estandarizar el contrato de respuesta bajo el patrón canónico Universal Envelope.
- **Fases de Implementación:**
  - **Fase 1 — Extracción del Servicio Estocástico GBM (ARQ-03):**
    - Archivo: `backend/app/domain/services/market_simulation.py`.
    - Extracción limpia de la simulación de precios fuera del router de backtest hacia un servicio puro de dominio.
  - **Fase 2 — Implementación del Universal Envelope (ARQ-02):**
    - Archivo: `backend/app/schemas/envelope.py`.
    - Contratos canónicos: BaseSchema (inmutable), ResponseMeta (request_id, timestamp, execution_time_ms), APIResponse[T] y APIErrorResponse (con códigos tipados y detalle granular).
  - **Fase 3 — Estandarización de Manejadores de Excepciones:**
    - Adaptación de manejadores HTTP 422 y 500 en main.py para responder bajo el formato canónico de APIErrorResponse.
  - **Fase 4 — Verificación y Testing (154 Tests en Verde):**
    - Creación de `backend/tests/unit/test_envelope.py` y validación de cobertura de simulación. Total: 154 tests en verde.

[IMPLEMENTADO] Módulo 3.5: Observabilidad y Telemetría de Producción (OBS-01, OBS-02)

- **Objetivo:** Implementar trazabilidad distribuida, correlación contextual de solicitudes y sondas de salud semánticas para orquestación de contenedores.
- **Fases de Implementación:**
  - **Fase 1 — Middleware ASGI de Correlación y Latencia (OBS-01):**
    - Archivo: `backend/app/api/middleware/trace.py`.
    - Middleware RequestTraceMiddleware: captura o generación de X-Request-ID almacenado en ContextVar, medición precisa de latencia con time.perf_counter() e inyección de encabezados de salida X-Request-ID y X-Execution-Time-Ms.
  - **Fase 2 — Sondas Semánticas de Salud (OBS-02):**
    - `/health/liveness`: verificación inmediata en memoria de la ejecución del event loop (retorno 200).
    - `/health/readiness`: comprobación asíncrona de conectividad con la base de datos (SELECT 1) con timeout acotado de 1.5s (retorno 200 o 503 degradado).
  - **Fase 3 — Verificación y Testing (159 Tests en Verde):**
    - Pruebas de middleware y endpoints de salud en test_trace_middleware.py y test_api_endpoints.py. Total: 159 tests en verde.

CAPÍTULO 4: Conectividad Externa, Ingesta en Vivo y Datos de Mercado

Estado: Completado. Transición desde datos simulados hacia la ingesta de mercado en vivo, arbitraje cambiario argentino (MEP/CCL), protección de red perimetral, persistencia de series OHLCV y adaptadores hacia backtesting.

[IMPLEMENTADO] Módulo 4.1: Contratos Canónicos y Calculadora FX (MEP/CCL)

- **Objetivo:** Definir contratos atómicos inmutables para series de precios aptas para cualquier representación visual (líneas, barras, velas, Heikin-Ashi) y modelar matemáticamente los tipos de cambio financieros implícitos con precisión Decimal y microestructura de puntas reales.
- **Fases de Implementación:**
  - **Fase 1 — Contratos DTO Inmutables (`backend/app/schemas/market_data.py`):**
    - Timeframe: Enum (1m, 5m, 15m, 1h, 1d, 1w).
    - FXType: Enum (MEP, CCL).
    - MarketQuoteDTO: ticker normalizado en mayúsculas, price: Decimal, bid/ask: Decimal | None, volume: Decimal y timestamp con zona horaria UTC estricta (offset cero obligatorio).
    - MarketBarDTO: estructura OHLCV atómica (open, high, low, close, volume, vwap).
  - **Fase 2 — Calculadora Cuantitativa Pura (`backend/app/domain/services/fx_calculator.py`):**
    - calculate_mep_rate(local_quote, usd_quote) → FXRateDTO:
      rate = local_quote.price / usd_quote.price
    - calculate_ccl_rate(local_quote, adr_quote, conversion_ratio) → FXRateDTO:
      rate = (local_quote.price * conversion_ratio) / adr_quote.price
    - Jerarquía de excepciones de dominio: FXCalculatorError, InvalidMarketPriceError, ZeroPriceDivisionError.
  - **Fase 3 — Hardening y Auditoría de Microestructura Bursátil:**
    - Validación geométrica de la barra: low <= min(open, close), high >= max(open, close) y acotamiento del VWAP: low <= vwap <= high.
    - Validación de zona horaria UTC estricta: rechazo de datetimes naive y de offsets distintos de cero en MarketQuoteDTO, MarketBarDTO y FXRateDTO.
    - Cuantización monetaria exacta con ROUND_HALF_UP: rate a 4 decimales (Decimal("0.0001")) y spread_pct a 2 decimales (Decimal("0.01")).

        - Microestructura cruzada de spread bid-ask:
          MEP Bid = local.bid / quote.ask, MEP Ask = local.ask / quote.bid,
          spread_pct = ((ask - bid) / rate) * 100
          Si alguna de las cuatro puntas falta o es <= 0, asignación segura de spread_pct = None.
    - **Fase 4 — Suite de Verificación (171 Tests en Verde):**
        - Creación de `backend/tests/unit/test_market_contracts.py` con pruebas de integridad geométrica, precisión decimal y casos límite de arbitraje cambiario.

[IMPLEMENTADO] Módulo 4.2: Conectores Asíncronos, Resiliencia y Blindaje SSRF

- **Objetivo:** Construir la infraestructura de red saliente hacia proveedores de mercado garantizando aislamiento frente a caídas de red, ataques de SSRF y DNS-Rebinding.
- **Fases de Implementación:**
    - **Fase 1 — Contrato de Proveedor Abstracto (`backend/app/domain/services/market_connector.py`):**
        - Protocolo BaseMarketConnector(ABC) con operaciones asíncronas get_quote y get_historical_bars.
        - Excepciones de dominio: MarketDataConnectorError, TickerNotFoundError, RateLimitExceededError, SecuritySSRFViolationError.
    - **Fase 2 — Guardián de Seguridad Perimetral Anti-SSRF (`backend/app/infrastructure/market_data/ssrf_guard.py`):**
        - Validación estricta de protocolos autorizados (http, https).
        - Bloqueo de resolución hacia infraestructura privada: loopback (127.0.0.0/8, ::1/128), redes locales RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), CGNAT (100.64.0.0/10), link-local / metadatos de nube (169.254.0.0/16, fe80::/10) y ULA IPv6 (fc00::/7).
        - Admisión de inyección de resolvedor DNS (dns_resolver) para posibilitar pruebas herméticas sin acceso a internet.
    - **Fase 3 — Cliente HTTP Asíncrono Resiliente (`backend/app/infrastructure/market_data/http_connector.py`):**
        - Conector HttpMarketConnector montado sobre httpx.AsyncClient.
        - Doble validación perimetral anti-SSRF: en la configuración inicial y antes de emitir cada petición al socket.
        - Mitigación de DNS-rebinding con follow_redirects=False.
        - Timeouts defensivos: conexión 3.0s, lectura 5.0s, escritura 3.0s.
        - Reintentos exponenciales con jitter (máximo 3) ante errores de conexión transitorios o códigos HTTP 429/503; parametrización de backoff_factor=0.0 en entornos de prueba para ejecución sin latencia.
    - **Fase 4 — Suite de Verificación (185 Tests en Verde):**

        - Creación de `backend/tests/unit/test_market_connector.py` con 14 pruebas unitarias herméticas mediante httpx.MockTransport.

[IMPLEMENTADO] Módulo 4.3: Persistencia Relacional, Deduplicación Delta y Caché Volátil

- **Objetivo:** Almacenar series temporales OHLCV de forma relacional, implementar ingesta incremental inteligente y desacoplar la carga mediante caché volátil.
- **Fases de Implementación:**
    - **Fase 1 — Modelo Relacional de Series Temporales (`backend/app/infrastructure/db/models/market_data.py`):**
        - Modelo MarketBarModel: tabla market_bars con columnas indexadas ticker, timeframe, timestamp, columnas de cotización Numeric(18, 4) y volumen Numeric(24, 4).
        - Constraint única física: UniqueConstraint("ticker", "timeframe", "timestamp", name="uq_market_bar_ticker_tf_timestamp").
        - Registro en base.py y models/**init**.py.
    - **Fase 2 — Caché en Memoria con Reloj Monótono (`backend/app/infrastructure/market_data/quote_cache.py`):**
        - Implementación de InMemoryQuoteCache: TTL parametrizado (30 segundos por defecto), temporizado mediante time.monotonic() para aislar la expiración de saltos horarios de reloj del sistema.
    - **Fase 3 — Servicio de Historial e Ingesta Delta (`backend/app/domain/services/market_history_service.py`):**
        - Servicio MarketHistoryService:
            - get_or_fetch_bars: evalúa la cobertura en base de datos; si faltan datos en los extremos del intervalo, consulta al conector externo; deduplica e inserta exclusivamente los registros faltantes vía session.add_all y session.flush.
            - get_quote_with_cache: consulta de baja latencia al caché volátil; delegación en conector y refresco de caché ante cache-miss.
            - Normalización defensiva de datetimes desde SQLite asegurando tzinfo=timezone.utc.
    - **Fase 4 — Suite de Verificación (190 Tests en Verde):**
        - Creación de `backend/tests/integration/test_market_persistence.py` con 5 pruebas de integración asíncrona sobre base de datos SQLite en memoria.

[IMPLEMENTADO] Módulo 4.4: Controladores REST (Universal Envelope) e Integración con Backtesting

- **Objetivo:** Exponer los servicios de mercado vía HTTP bajo el estándar canónico y conectar las series de precios reales con el motor de backtesting.
- **Fases de Implementación:**
    - **Fase 1 — Inyección de Dependencias y Rutas:**
        - Archivo `backend/app/api/deps.py`: singleton global_quote_cache y proveedor get_market_service(db).
        - Archivo `backend/app/core/config.py`: parámetro MARKET_DATA_BASE_URL.
        - Archivo `backend/app/api/v1/endpoints/market.py`:
            - GET /quote/{ticker}: validación regex ^[A-Z0-9.-]+$ (422 ante formato inválido), resolución con caché y retorno APIResponse[MarketQuoteDTO].
            - GET /history/{ticker}: validación cronológica (start_date <= end_date), ingesta incremental y retorno APIResponse[list[MarketBarDTO]].
            - GET /fx/mep y GET /fx/ccl: cómputo y retorno estructurado de tasas y spreads en APIResponse[FXRateDTO].
        - Montaje formal en `backend/app/api/v1/router.py` bajo el prefijo /market.
    - **Fase 2 — Adaptador Cuantitativo para Backtesting:**
        - Archivo `backend/app/domain/services/backtest_engine.py`: función bars_to_price_history(bars_by_ticker) que transforma barras canónicas MarketBarDTO en matrices de precios de cierre {ticker: {date: close}}, permitiendo simular estrategias sobre datos históricos de mercado reales sin alterar interfaces preexistentes.
    - **Fase 3 — Suite de Verificación (205 Tests en Verde):**
        - Creación de `backend/tests/integration/test_api_market.py` con 14 pruebas de integración HTTP cubriendo respuestas exitosas, códigos 404/422/429 tipados y ejecución de backtesting con datos reales.
        - Resultado consolidado: 205 pruebas ejecutadas al 100% en verde (165 unitarias + 40 de integración).

CAPÍTULO 5: Seguridad Criptográfica, Autenticación y Control de Acceso

Estado: Pendiente. Resolución de la vulnerabilidad crítica SEC-03 del informe de auditoría: erradicación total del mock de autenticación basado en X-User-Id y mitigación de vulnerabilidades IDOR/BOLA.

[PENDIENTE] Módulo 5.1: Modelo de Identidad, Hashing Criptográfico y Gestión de Secretos

- **Objetivo:** Establecer el almacenamiento seguro de credenciales con algoritmos resistentes a ataques de fuerza bruta y colisiones.
- **Fases de Implementación:**
    - **Fase 1 — Modelo Relacional de Usuario (UserModel):**
        - Archivo: `backend/app/infrastructure/db/models/user.py`.
        - Columnas: id (UUIDv4), email (String único indexado), hashed_password (String), full_name (String), is_active (Boolean), role (Enum UserRole: VIEWER, TRADER, ADMIN), created_at y updated_at.
    - **Fase 2 — Servicio de Hashing Defensivo:**
        - Archivo: `backend/app/core/security.py`.
        - Derivación de claves con Argon2id o bcrypt con factores de costo elevados contra aceleración por hardware.
        - Funciones: verify_password y get_password_hash.
        - Tipado estricto en esquemas Pydantic mediante SecretStr para proteger contraseñas en memoria.

[PENDIENTE] Módulo 5.2: Motor de Tokens Criptográficos (JWT / OAuth2 Bearer)

- **Objetivo:** Emitir y validar tokens de acceso y refresco firmados digitalmente para autenticación sin estado de alta seguridad.
- **Fases de Implementación:**
    - **Fase 1 — Generación y Firma de Tokens JWT:**
        - Archivo: `backend/app/core/security.py`.
        - Firma simétrica robusta (HMAC-SHA256 con clave secreta de alta entropía) o asimétrica (Ed25519).
        - Claims canónicos: sub (User ID), exp (expiración corta: 15-30 min), nbf, iat, role y jti (identificador único para listas de revocación).
    - **Fase 2 — Flujo de Refresco de Tokens (Refresh Tokens):**
        - Modelo de tokens de refresco firmados con vida extendida (7 a 30 días) para renovar el token de acceso sin reingreso de credenciales.

[PENDIENTE] Módulo 5.3: Dependencias de Seguridad, RBAC y Aislamiento Anti-IDOR (SEC-03)

- **Objetivo:** Reemplazar el mock en deps.py e imponer autorización por roles y aislamiento multitenant estricto en toda la API.
- **Fases de Implementación:**
    - **Fase 1 — Extracción y Validación del Token Bearer:**
        - Archivo: `backend/app/api/deps.py`.

        - Reemplazo del mock request.headers.get("X-User-Id") por OAuth2PasswordBearer.
        - Dependencia get_current_user: decodificación de token, verificación criptográfica de firma, validación de expiración y resolución de la entidad de usuario en base de datos (retorno de HTTP 401 estructurado ante fallos).
    - **Fase 2 — Control de Acceso Basado en Roles (RBAC):**
        - Dependencia RoleChecker(allowed_roles=[...]).
        - Restricción de endpoints sensibles (creación de órdenes, ejecución de rebalanceos, ingesta forzada de mercado) según el rol del usuario.
    - **Fase 3 — Aislamiento Multitenant Estricto (Mitigación IDOR/BOLA):**
        - Adaptación de endpoints de portafolios (/portfolios/*): forzar que toda operación de consulta o mutación filtre mandatoriamente por el user_id criptográficamente verificado del token. Prohibición de acceso a recursos de terceros.

[PENDIENTE] Módulo 5.4: Endpoints REST de Autenticación y Suite de Pruebas

- **Objetivo:** Exponer el registro, inicio de sesión y renovación de credenciales bajo el patrón Universal Envelope y verificar la seguridad con tests de penetración.
- **Fases de Implementación:**
    - **Fase 1 — Endpoints de Autenticación:**
        - Archivo: `backend/app/api/v1/endpoints/auth.py`.
        - POST /api/v1/auth/register: validación de contraseña fuerte, registro de usuario y retorno de APIResponse[UserResponse].
        - POST /api/v1/auth/login: validación de credenciales y retorno de APIResponse[TokenResponse].
        - POST /api/v1/auth/refresh: renovación de access token.
    - **Fase 2 — Suite de Pruebas de Seguridad y Fixtures:**
        - Archivo: `backend/tests/integration/test_auth_and_rbac.py`.
        - Fixtures en conftest.py: clientes autenticados (authenticated_client, admin_client).
        - Pruebas adversariales de denegación: tokens manipulados, firmas alteradas, tokens expirados (401/403).
        - Pruebas de intrusión IDOR: validación de que el Usuario A no pueda acceder ni mutar recursos pertenecientes al Usuario B.

CAPÍTULO 6: Ingesta de Reportes Locales y Pipeline de Calidad de Datos

Estado: Pendiente. Cumplimiento directo de los requerimientos de trabajo de campo relevados en la Etapa 1: extracción, saneamiento y normalización de planillas de inversión (CSV/Excel) de ALyCs y plataformas locales.

[PENDIENTE] Módulo 6.1: Parser Desacoplado de Extractos y Reportes de ALyCs (CSV / Excel)

- **Objetivo:** Ingerir y deserializar de forma resiliente planillas de saldos, tenencias y boletos de liquidación provenientes de diversos brokers locales (ej. Balanz, IOL, PPI, Bull Market, Caja de Valores).
- **Fases de Implementación:**
    - **Fase 1 — Interfaz Abstracta de Parsing y Estrategias por Formato:**
        - Archivo: `backend/app/domain/services/ingestion/base_parser.py`.
        - Protocolo abstracto BaseReportParser(ABC) con método async def parse_report(file_stream: bytes, filename: str) → list[RawHoldingRecord].
        - Implementación de estrategias especializadas en excel_parser.py (usando openpyxl en modo read-only para optimizar memoria) y csv_parser.py.
    - **Fase 2 — Detección Automática de Esquemas y Normalización de Columnas:**
        - Archivo: `backend/app/domain/services/ingestion/schema_detector.py`.
        - Algoritmo de mapeo difuso (fuzzy matching) de encabezados para normalizar columnas arbitrarias de brokers (Especie, Ticker, Símbolo, Cantidad, Nominales, Precio Promedio, Moneda de Liquidación).
    - **Fase 3 — Suite de Pruebas con Muestras Reales:**
        - Archivo: `backend/tests/unit/test_report_parsers.py`.
        - Pruebas con archivos sintéticos simulando layouts reales de ALyCs, archivos malformados, codificaciones problemáticas (UTF-8, Latin-1, Windows-1252) y detección de errores descriptivos.

[PENDIENTE] Módulo 6.2: Pipeline de Validación, Normalización y Tratamiento Contable

- **Objetivo:** Sanear inconsistencias contables detectadas en el trabajo de campo (empresas con EPS <= 0, descalces de moneda ARS/USD y ratios comprometidos).
- **Fases de Implementación:**
    - **Fase 1 — Pipeline de Saneamiento y Validación de Esquemas:**

        - Archivo: `backend/app/domain/services/ingestion/data_cleaning_pipeline.py`.
        - Validación de tipos de datos, limpieza de caracteres monetarios ($, USD, ,, .), detección de nominales negativos o nulos y normalización de tickers locales (agregando o depurando sufijos .BA).
    - **Fase 2 — Manejo Robusto de Inconsistencias de Balance (EPS <= 0):**
        - Archivo: `backend/app/domain/services/accounting_integrity.py`.
        - Implementación de la directiva del trabajo de campo: etiquetado determinista de ratios patológicos (P/E, PEG) como INDETERMINATE ante utilidades netas negativas o nulas, impidiendo interrupciones por división por cero y emitiendo alertas cuantitativas al inversor.
    - **Fase 3 — Suite de Pruebas de Calidad de Datos:**
        - Archivo: `backend/tests/unit/test_accounting_integrity.py`.
        - Pruebas de estrés sobre balances con patrimonio neto negativo, empresas en etapa inicial con pérdidas operativas y activos deslistados.

[PENDIENTE] Módulo 6.3: Integración Transaccional y Conciliación Automática de Tenencias

- **Objetivo:** Exponer la carga de archivos vía API REST y actualizar el portafolio del usuario de forma atómica.
- **Fases de Implementación:**
    - **Fase 1 — Servicio de Conciliación de Cartera (PortfolioReconciliationService):**
        - Archivo: `backend/app/domain/services/portfolio_reconciliation.py`.
        - Comparación de las tenencias persistidas en base de datos versus los registros extraídos del extracto bancario/ALyC.
        - Generación de eventos de ajuste de inventario o reemplazo total de snapshot con trazabilidad contable.
    - **Fase 2 — Controlador REST de Ingesta:**
        - Archivo: `backend/app/api/v1/endpoints/ingestion.py`.
        - POST /api/v1/ingestion/upload-report: subida multiparte (UploadFile), validación de tipos MIME autorizados (.csv, .xlsx, .xls), límite de tamaño de archivo (máx 10 MB) y procesamiento en pipeline.
        - Respuesta envuelta en APIResponse[IngestionReportSummary].
    - **Fase 3 — Pruebas de Integración E2E:**
        - Archivo: `backend/tests/integration/test_api_ingestion.py`.
        - Simulación de carga de extractos de brokers sobre el cliente asíncrono y verificación de la actualización persistida en base de datos.

CAPÍTULO 7: Motores Cuantitativos Avanzados, Señales y Valuación Mark-to-Market

Estado: Pendiente. Ampliación de capacidades analíticas superiores, rebalanceo disciplinado y cálculo de rendimiento patrimonial multimoneda en tiempo real.

[PENDIENTE] Módulo 7.1: Indicadores Técnicos y Generador de Señales Cuantitativas

- **Objetivo:** Computar métricas analíticas deterministas sobre series MarketBarDTO para alimentar estrategias algorítmicas.
- **Fases de Implementación:**
    - **Fase 1 — Biblioteca de Indicadores Numéricos Vectorizados:**
        - Archivo: `backend/app/domain/services/technical_indicators.py`.
        - Implementación sin dependencias externas pesadas: Medias Móviles Simples y Exponenciales (SMA/EMA), Índice de Fuerza Relativa (RSI de Wilder), Bandas de Bollinger y Rango Verdadero Promedio (ATR).
    - **Fase 2 — Motor de Generación de Señales de Negociación:**
        - Archivo: `backend/app/domain/services/signal_generator.py`.
        - Reglas de entrada y salida basadas en convergencia de indicadores y volatilidad realizada, emitiendo órdenes estructuradas hacia la FSM transaccional.
    - **Fase 3 — Pruebas Unitarias de Precisión:**
        - Archivo: `backend/tests/unit/test_technical_indicators.py`.
        - Verificación contra series de benchmark cuantitativo precalculadas.

[PENDIENTE] Módulo 7.2: Valuación Patrimonial Dinámica y PnL Multimoneda (TWR / MWR)

- **Objetivo:** Computar el Valor Cuota y el Patrimonio Neto (NAV) en tiempo real, reflejando tenencias bimoneda (ARS/USD) convertidas mediante el tipo de cambio Dólar MEP o CCL.
- **Fases de Implementación:**
    - **Fase 1 — Motor de Valuación Mark-to-Market Bimoneda:**
        - Archivo: `backend/app/domain/services/portfolio_valuation_service.py`.

        - Valuación dinámica: activos en pesos valuados a último precio local; activos en moneda extranjera o CEDEARs valuados en dólares utilizando el tipo de cambio financiero en tiempo real provisto por fx_calculator.py.
        - Desglose de PnL latente (no realizado) y PnL realizado con trazabilidad de comisiones e IVA.
    - **Fase 2 — Métricas de Rendimiento Ponderado:**
        - Retorno Ponderado por Tiempo (Time-Weighted Return - TWR): aislamiento estricto de flujos de caja externos (suscripciones y rescates) para evaluar la habilidad del gestor.
        - Retorno Ponderado por Dinero (Money-Weighted Return - MWR / TIR interna de la cartera).
    - **Fase 3 — Pruebas Unitarias y de Integración:**
        - Archivo: `backend/tests/unit/test_portfolio_valuation.py`.
        - Casos de prueba bimoneda con saltos cambiarios y cálculo de rendimientos ante depósitos intermedios.

[PENDIENTE] Módulo 7.3: Rebalanceo por Bandas de Tolerancia Dinámicas

- **Objetivo:** Cumplir el requerimiento del trabajo de campo: mitigar la deriva de cartera (portfolio drift) mediante rebalanceo por umbrales relativos y absolutos (Tolerance-Band Rebalancing).
- **Fases de Implementación:**
    - **Fase 1 — Algoritmo de Bandas de Tolerancia:**
        - Archivo: `backend/app/domain/services/tolerance_band_rebalancer.py`.
        - Monitoreo de desvíos |w_i(t) - w_i^*| > theta_i. La banda theta_i se ajusta dinámicamente en función de la volatilidad del activo sigma_i y los costos de transacción locales de la ALyC (Módulo 1.3). |---|Órdenes Óptimas de Rebalanceo:**
        - Determinación de las compras y ventas mínimas indispensables para devolver los activos dentro del corredor de tolerancia sin incurrir en rotación excesiva (portfolio turnover).
    - **Fase 3 — Pruebas de Simulación de Deriva:**
        - Archivo: `backend/tests/unit/test_tolerance_rebalancing.py`.
        - Comparación de desempeño entre rebalanceo por calendario fijo vs. rebalanceo por bandas de tolerancia ante costos reales de mercado.

CAPÍTULO 8: Ciclo de Vida de Base de Datos y Automatización en Background

Estado: Pendiente. Estandarización de migraciones estructuradas con Alembic, automatización de ingesta bursátil periódica y métricas de producción.

[PENDIENTE] Módulo 8.1: Versionado y Migraciones Estructuradas con Alembic

- **Objetivo:** Establecer un pipeline determinista de evolución de esquemas relacionales para PostgreSQL en producción.
- **Fases de Implementación:**
    - **Fase 1 — Inicialización y Parametrización de Alembic:**
        - Archivo: `alembic.ini` y `backend/migrations/env.py`.
        - Configuración de ejecución asíncrona (run_async_migrations) conectada a settings.DATABASE_URI y apuntando a los metadatos unificados Base.metadata.
    - **Fase 2 — Generación de Migraciones Base:**
        - Creación de la migración inicial consolidada con todas las tablas del sistema: portfolios, holdings, orders, market_bars y users.
    - **Fase 3 — Pruebas de Migración y Rollback:**
        - Validación de scripts de migración hacia adelante (upgrade) y hacia atrás (downgrade) en entorno de CI.

[PENDIENTE] Módulo 8.2: Schedulers de Ingesta Bursátil Programada y Bloqueo Concurrente

- **Objetivo:** Automatizar la descarga diaria de precios de cierre al finalizar cada rueda operativa bursátil sin intervención manual ni bloqueo de endpoints HTTP.
- **Fases de Implementación:**
    - **Fase 1 — Orquestador de Tareas Programadas:**
        - Archivo: `backend/app/infrastructure/tasks/market_scheduler.py`.
        - Programación de tareas de fondo (utilizando APScheduler o Celery asíncrono) para ejecutarse de lunes a viernes al cierre de los mercados (17:30 ART / 16:30 EDT).
        - Invocación automática de MarketHistoryService.get_or_fetch_bars para mantener la base de datos actualizada incrementalmente.
    - **Fase 2 — Bloqueos Distribuidos y Concurrencia:**
        - Implementación de locks basados en Redis o base de datos relacional para evitar que múltiples réplicas del backend intenten descargar simultáneamente el mismo activo.

[PENDIENTE] Módulo 8.3: Monitoreo Institucional y Métricas RED (Prometheus / Grafana)

- **Objetivo:** Exportar métricas de salud de infraestructura y computación cuantitativa para observabilidad en tiempo real.
- **Fases de Implementación:**

    - **Fase 1 — Instrumentación del Endpoint /metrics:**
        - Archivo: `backend/app/api/middleware/metrics.py`.
        - Exportación de métricas RED: Rate (tasa de peticiones por endpoint), Errors (contadores de códigos 4xx/5xx) y Duration (histogramas de latencia).
    - **Fase 2 — Métricas de Dominio Cuantitativo:**
        - Latencia de optimización de carteras (SLSQP vs Spinu), tasa de fallbacks activados y métricas de aciertos de caché (cache hit ratio) en cotizaciones.

CAPÍTULO 9: Frontend Dashboard Reactivo (React + Vite + TypeScript)

Estado: Pendiente. Construcción de la interfaz de usuario interactiva planteada en la Etapa 1 para visualización patrimonial, monitoreo de riesgo y carga de extractos.

[PENDIENTE] Módulo 9.1: Arquitectura Base, Design System y Contratos DTO Tipados

- **Objetivo:** Inicializar el entorno frontend con arquitectura limpia, contratos de datos estrictos sincronizados con FastAPI y un sistema de diseño profesional.
- **Fases de Implementación:**
    - **Fase 1 — Inicialización del Entorno de Desarrollo:**
        - Proyecto: `frontend/` configurado con React 18+, Vite, TypeScript estricto y Tailwind CSS.
        - Cliente HTTP centralizado (axios o ky) configurado con interceptores para inyectar automáticamente Authorization: Bearer y capturar el encabezado X-Request-ID.
    - **Fase 2 — Sincronización de Contratos Tipados:**
        - Generación o tipado espejo de los DTOs de backend (APIResponse, MarketQuoteDTO, MarketBarDTO, FXRateDTO, PortfolioDTO).
    - **Fase 3 — Shell de la Aplicación y Autenticación:**
        - Páginas de Login y Registro conectadas a los endpoints del Capítulo 5.
        - Gestión de estado de sesión (Zustand o Context API) con almacenamiento seguro de tokens.

[PENDIENTE] Módulo 9.2: Visualización Financiera Avanzada (TradingView Lightweight Charts)

- **Objetivo:** Brindar gráficos financieros interactivos de alto rendimiento consumiendo las series temporales canónicas OHLCV.
- **Fases de Implementación:**

    - **Fase 1 — Integración de TradingView Lightweight Charts:**
        - Componente reutilizable CandleChart: renderizado de velas japonesas, gráficos de líneas y áreas a partir del contrato canónico MarketBarDTO.
    - **Fase 2 — Visualizador de Arbitraje Cambiario (MEP / CCL):**
        - Panel en vivo que muestra la evolución de los tipos de cambio financieros implícitos, cotizaciones del bono AL30/GD30 en ambas monedas y spread bid-ask porcentual.
    - **Fase 3 — Métricas de Riesgo y Curvas de Backtesting:**
        - Gráficos interactivos de evolución de patrimonio, áreas de drawdown histórico y comparación de retornos versus el índice S&P 500 o Merval.

[PENDIENTE] Módulo 9.3: Centro de Carga de Extractos, Monitoreo de Riesgo y Gestión de Carteras

- **Objetivo:** Interfaz gráfica para la operativa diaria del inversor y asesor patrimonial según los requerimientos de campo.
- **Fases de Implementación:**
    - **Fase 1 — Dropzone de Carga de Extractos (CSV/Excel):**
        - Componente interactivo para arrastrar y soltar reportes de ALyCs, visualizando barras de progreso y reportes de normalización de datos.
    - **Fase 2 — Matriz de Cumplimiento de Reglas de Riesgo (Semaforización):**
        - Tablero de control interactivo que visualiza el estado de las reglas del RiskEngine relevadas en campo (Concentración Máxima, Mínimo en Renta Fija, Anomalías contables de EPS <= 0, y Deriva de cartera) con semáforos verdes, amarillos y rojos.
    - **Fase 3 — Panel de Órdenes y Rebalanceo Asistido:**
        - Visualización de órdenes de rebalanceo sugeridas (compra/venta de nominales) con cálculo previo de comisiones e impuestos de mercado antes de su confirmación final.

Matriz Consolidada de Estado y Secuencia de Ejecución

Capítulo	Título	Módulos	Estado	Dependencias Clave
Capítulo 1	Fundaciones Cuantitativas y Arquitectura Base	1.1 al 1.5	IMPLEMENTADO	Ninguna (Núcleo)
Capítulo 2	Auditoría Integral y Diagnóstico Técnico	2.1 y 2.2	IMPLEMENTADO	Cap. 1

Capítulo	Título	Módulos	Estado	Dependencias Clave
Capítulo 3	Remediación Técnica y Hardening Post-Auditoría	3.1 al 3.5	IMPLEMENTADO	Cap. 2 (AUDIT_REPORT.md)
Capítulo 4	Conectividad Externa, Ingesta en Vivo y Mercado	4.1 al 4.4	IMPLEMENTADO	Cap. 1, Cap. 3
Capítulo 5	Seguridad Criptográfica, Autenticación y RBAC	5.1 al 5.4	PENDIENTE (Inmediato)	Cap. 3 (SEC-03)
Capítulo 6	Ingesta de Reportes Locales y Calidad de Datos	6.1 al 6.3	PENDIENTE	Cap. 1, Cap. 5
Capítulo 7	Motores Cuantitativos Avanzados y Valuación	7.1 al 7.3	PENDIENTE	Cap. 4, Cap. 6
Capítulo 8	Ciclo de Vida de DB (Alembic) y Background Tasks	8.1 al 8.3	PENDIENTE	Cap. 4, Cap. 5
Capítulo 9	Frontend Dashboard Reactivo (React + Vite)	9.1 al 9.3	PENDIENTE	Cap. 4, Cap. 5, Cap. 6